



CC1002 - Introducción a la Programación

MÓDULO 2: DISEÑO Y USO DE FUNCIONES

Departamento de Ciencias de la Computación (DCC) - FCFM
UNIVERSIDAD DE CHILE



Eduardo Godoy (egodoy@dcc.uchile.cl)

Nadia Decar (nadiadecar@ug.uchile.cl)

(basado en apuntes del profesor P. Inostroza)



Solución del ejercicio anterior

ENUNCIADO

Dado un número de 6 dígitos, saque el 4^{to} dígito

– Indicación: use //, % y variables auxiliares

Idea de la solución

1. Con // se separan los dígitos de la izquierda
2. Con % se guardan los 3 dígitos finales
3. Se reconstruye el número sin ese dígito

SESIÓN EN PYTHON

```
>>>numero = 129345 # cualquier número de 6  
          dígitos
```

```
>>>superior = numero // 10000
```

```
>>>superior  
      ⇒12
```

```
>>>resto = numero % 1000
```

```
>>>resto  
      ⇒345
```

```
>>>numero = superior * 1000 + resto
```

```
>>>numero  
      ⇒12345
```



HOUDINI

DUA LIPA

String

Operando con texto

```
>>> texto = 'abracadabra'  
>>> texto[0:4]  
⇒ 'abra'  
>>> texto * 2  
⇒ 'abracadabraabracadabra'
```



String $\leftrightarrow$ Entero

`int(texto)`

Dado un texto, retorna un entero

```
>>>int('100') + 1  
⇒101
```

```
>>>int('7') * 3  
⇒21
```

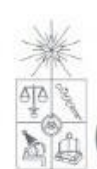
`str(valor)`

Dado un valor, retorna un texto

```
>>>str(100) + '1'  
⇒1001
```

```
>>>'Hay ' + str(100) + ' alumnos'  
⇒'Hay 100 alumnos'
```

Ojo: `'Hay ' + 100` produce un **TypeError**: no se puede mezclar texto y números sin convertir.



String

`texto[ inicio : fin : paso ]`

REBANADAS HACIA ADELANTE

```
>>>texto = 'abracadabra'
```

```
>>>texto
```

```
⇒'abracadabra'
```

```
>>>texto[ 2: 6 ]
```

```
⇒'raca'
```

```
>>>texto[ 2: ]
```

```
⇒'racadabra'
```

```
>>>texto[ : 6 ]
```

```
⇒'abraca'
```

```
>>>texto[ 1: 10: 2 ]
```

```
⇒'baaar'
```

CON PASO NEGATIVO

```
>>>texto = 'abracadabra'
```

```
>>>texto
```

```
⇒'abracadabra'
```

```
>>>texto[ 6: 2 ]
```

```
⇒''
```

```
>>>texto[ 6: 2: -1 ]
```

```
⇒'daca'
```

```
>>>texto[ 10: 0: -1 ]
```

```
⇒'arbadacarb'
```

```
>>>texto[ 10 : : -1 ]
```

```
⇒'arbadacarba'
```



FUNCIONES

Recuerdos de cc1000

```
>>>def areaCirculo(radio):  
    return 3.14 * radio ** 2  
>>>areaCirculo(10)  
    ⇒314.0
```



Tareas que se repiten

Calcule el área de un círculo de radio

10

– use $\pi = 3.14$

```
>>>radio1 = 10
>>>pi = 3.14
>>>area = pi * radio1 ** 2
>>>area
⇒314.0
```

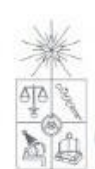
Calcule el área de un círculo de radio

20

– use $\pi = 3.14$

```
>>>radio2 = 20
>>>pi = 3.14
>>>area = pi * radio2 ** 2
>>>area
⇒1256.0
```

El cálculo es el mismo: solo cambia el radio. Repetir código es señal de que falta una **función**.



Función: Agrupando tareas que se repiten

DEFINICIÓN

```
>>>def areaCirculo(radius):  
    pi = 3.14  
    return pi * radius ** 2
```

USO

```
>>>areaCirculo(10)  
    ⇒314.0  
  
>>>areaCirculo(20)  
    ⇒1256.0
```

Una sola definición, muchos usos: el parámetro `radius` toma el valor del argumento en cada llamada.



Función: Agrupando tareas que se repiten

DEFINICIÓN CORRECTA

```
>>>def areaCirculo(radio):  
    pi = 3.14  
    return pi * radio ** 2
```

LLAMADA CON OTRO NOMBRE

```
>>>areacirculo(10)  
Traceback (most recent call last):  
  File "<stdin>", line 1, in  
<module>  
NameError: name 'areacirculo'  
is not defined
```

Python diferencia mayúsculas de minúsculas: el nombre debe escribirse exactamente igual que en la definición.



Función: Agrupando tareas que se repiten

SIN INDENTAR

```
>>>def areaCirculo(radius):  
>>>pi = 3.14
```

INDENTATIONERROR

```
File "<stdin>", line 2  
  pi = 3.14  
  ^
```

IndentationError: expected
an indented block

Python exige indentación: el cuerpo de la función va desplazado hacia la derecha (una sangría o Tab).



Función: Agrupando tareas que se repiten

ASÍ NO: SIN SANGRÍA

```
>>> def areaCirculo(radio):
```

```
>>> pi = 3.14
```

```
File "<stdin>", line 2
```

```
    pi = 3.14
```

```
    ^
```

IndentationError: expected an
indented block

ASÍ SÍ: CUERPO INDENTADO

```
>>> def areaCirculo(radio):
```

```
    pi = 3.14
```

```
    return pi * radio ** 2
```

```
    ↳ sangría (Tab)
```

Python exige indentación (“Tab”): la sangría es la que delimita el cuerpo de la función.



Funciones: Ejemplo

Área entre dos círculos

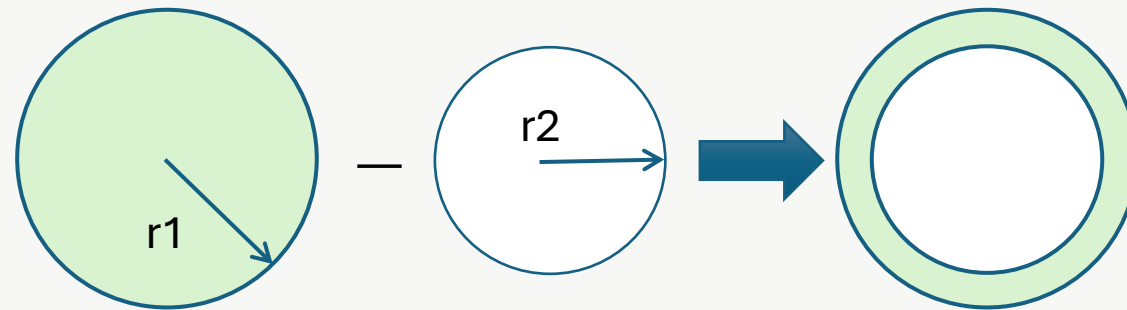
```
>>>areaCirculo(2)  
⇒12.56  
>>>areaAnillo(2, 1)  
⇒9.42
```



Área entre dos círculos

Construir la función **areaAnillo**, que calcula el área del anillo formado por 2 círculos (diferencia entre las áreas) de radios $r1$ y $r2$.

Restricción: $r1 \geq r2$



Función areaCirculo

```
# areaCirculo: num -> float
# calcula area de un circulo de radio radio
# ejemplo: areaCirculo(1) entrega 3.14
def areaCirculo(radio):
    pi = 3.14
    return pi * radio ** 2

# Test
assert areaCirculo(1) == 3.14
```

LA RECETA

1. Contrato

tipos de entrada y salida

2. Propósito

qué calcula la función

3. Ejemplo

un caso concreto

4. Código

def + cuerpo indentado

5. Test

assert que lo verifica



Función areaAnillo

```
# areaAnillo: num num -> float
# calcula area de anillo formado por
# dos circulos de radios rMayor y rMenor
# ej: areaAnillo(2,1) entrega 9.42
def areaAnillo(rMayor, rMenor):
    assert rMayor >= rMenor
    return areaCirculo(rMayor) - areaCirculo(rMenor)

# Test
assert areaAnillo(2,1) == 9.42
```

QUÉ CAMBIA AQUÍ

Dos parámetros

el contrato ahora es num num -> float

assert de restricción

obliga a que $rMayor \geq rMenor$

Reutiliza areaCirculo

no repite el cálculo del área

Test con dos valores

`assert areaAnillo(2,1) == 9.42`



Función areaAnillo

LA QUE LLAMA

```
# areaAnillo: num num -> float
# calcula area de anillo formado por
# dos circulos de radios rMayor y rMenor
# ej: areaAnillo(2,1) entrega 9.42
def areaAnillo(rMayor, rMenor):
    assert rMayor >= rMenor
    return areaCirculo(rMayor) - areaCirculo(rMenor)

# Test
assert areaAnillo(2,1) == 9.42
```

LA QUE ES LLAMADA

```
# areaCirculo: num -> float
# calcula area de un circulo
# de radio radio
# ejemplo: areaCirculo(1)
#           entrega 3.14
def areaCirculo(radio):
    pi = 3.14
    return pi * radio ** 2
```

IMPORTANTE: una función puede hacer uso de otra función.



Funciones: Receta

Cómo diseñar una función

```
# areaRectangulo: num num -> num
# calcula el area de un
# rectangulo
# ej: areaRectangulo(5, 3)
# entrega 15
def areaRectangulo(largo, ancho):
    return largo * ancho
assert areaRectangulo(5, 3) == 15
```



Receta de diseño

Toda función debe explicar qué hace

- Para qué está hecha
- Cuál o cuáles son sus entradas
- Cuál es su salida
- Un ejemplo explicativo
- Hay que comprobar que hace lo correcto

IMPORTANTE: si su tarea o control no viene con receta de diseño, su nota bajará notoriamente.



La función con toda la receta

PROBLEMA

Diseñar una función que calcule el área de un rectángulo.

SOLUCIÓN INCOMPLETA

```
def areaRectangulo(largo, ancho):  
    return largo * ancho
```

Falta la receta de diseño: el código funciona, pero no dice qué consume, qué produce ni cómo se prueba.



Receta de diseño: Detalles

Por el momento, **4 actividades**:

1

Entender el
propósito de la
función

2

Dar ejemplos de uso
de la función

3

Probar la función

4

Especificar el cuerpo
de la función



1. Entender el propósito de la función

1

Dar un nombre a la función

¡Que sea representativo!

2

Indicar qué consume

Qué recibe: sus parámetros

3

Indicar lo que produce

Qué retorna

Contrato: ¡los tres pasos anteriores juntos! Por lo general es la primera línea de la receta.



1. Entender el propósito de la función

PROBLEMA

Diseñar una función que calcule el área de un rectángulo.

EL CONTRATO

```
# areaRectangulo:  
#   num num -> num
```

Se lee así: la función `areaRectangulo` consume dos números y produce un número.



1. Entender el propósito de la función

LOS TRES PASOS

1. Dar un nombre

¡Que sea representativo!

2. Indicar qué consume

¿Qué recibe?, sus parámetros

3. Indicar lo que produce

¿Qué retorna?

EL CONTRATO RESULTANTE

areaRectangulo:

num num -> num

Cada color corresponde a uno de los tres pasos de la izquierda.



1. Entender el propósito de la función

Qué consume / qué produce: los tipos que usamos en el contrato.

int

Tipo entero

float

Tipo real

num

Tipo numérico
(real o entero)

str

Tipo texto

boolean

Tipo lógico

En el contrato: `# areaRectangulo: num num -> num`



1. Entender el propósito de la función

ENCABEZADO DE LA FUNCIÓN

```
# areaRectangulo: num num -> num
```

```
def areaRectangulo(largo, ancho) :
```

def

Define el inicio de la función

areaRectangulo

Nombre de la función

(largo, ancho)

Parámetros de la función

:

Los dos puntos
son obligatorios



1. Entender el propósito de la función

COMENTAR QUÉ HACE LA FUNCIÓN

```
# areaRectangulo: num num -> num  
# calcula el area de un rectangulo  
# de medidas largo y ancho  
def areaRectangulo(largo, ancho):
```

PROPÓSITO

Una o dos líneas que explican, en palabras, qué calcula la función.

Va justo debajo del contrato, también como comentario.



Receta de diseño: Detalles

Por el momento, **4 actividades**:

1

Entender el propósito
de la función

2

**Dar ejemplos de uso
de la función**

3

Probar la función

4

Especificar el cuerpo
de la función

2. Dar ejemplos de uso de la función

DAR UN EJEMPLO DE USO

```
# areaRectangulo: num num -> num
# calcula el area de un rectangulo
# de medidas largo y ancho
# ejemplo: areaRectangulo(5, 3) debe
#           producir 15
def areaRectangulo(largo, ancho):
```

EL EJEMPLO

Debe aportar al entendimiento de la función.

Con valores significativos, no con 0 ni 1.



Receta de diseño: Detalles

Por el momento, **4 actividades**:

1

Entender el propósito
de la función

2

Dar ejemplos de uso
de la función

3

Probar la función

4

Especificar el cuerpo
de la función



3. Probar la función

Testing

Proceso de probar una función.

Test

Una prueba concreta.

assert

La instrucción con la que se escribe un test.

Si la prueba está bien: es transparente, no pasa nada. **Si la prueba falla:** se genera un error.

Por lo general la prueba es representativa, con un valor calculado “a mano”.



3. Probar la función

SI LA PRUEBA ESTÁ BIEN

```
assert areaRectangulo(5, 3) == 15
```

Es transparente: no ocurre nada, la sesión sigue igual.

SI LA PRUEBA FALLA

```
assert areaRectangulo(5, 3) == 0
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

AssertionError

Se genera un error: la prueba avisa que algo está mal.



3. Probar la función

LA PRUEBA DENTRO DE LA RECETA

```
# areaRectangulo: num num -> num
# calcula el area de un rectangulo
# de medidas largo y ancho
# ejemplo: areaRectangulo(5, 3) debe
#           producir 15
def areaRectangulo(largo, ancho):

# Test
assert areaRectangulo(5, 3) == 15
```

POR QUÉ SIRVE

El test queda escrito junto a la función: cada vez que la ejecutes, se comprueba sola. El valor esperado sale del ejemplo, calculado a mano.



Receta de diseño: Detalles

Por el momento, **4 actividades**:

1

Entender el propósito
de la función

2

Dar ejemplos de uso
de la función

3

Probar la función

4

**Especificar el cuerpo
de la función**

4. Especificar el cuerpo de la función

PARA FORMULAR EL CUERPO HAY QUE ENTENDER

1. Los parámetros / valores de entrada
- 2.Cuál es la salida que debe entregar
3. Cómo, usando la entrada, se genera la salida

LA SALIDA PUEDE SER

- El resultado de una operación matemática
- El tratamiento de una secuencia de palabras

Siempre se entrega con **return**.



La función con toda la receta

```
# areaRectangulo: num num -> num
# calcula el area de un rectangulo
# de medidas largo y ancho
# ejemplo: areaRectangulo(5, 3) debe
#           producir 15
def areaRectangulo(largo, ancho):
    return largo * ancho

# Test
assert areaRectangulo(5, 3) == 15
```

LA RECETA COMPLETA

1. Contrato

línea 1

2. Propósito

líneas 2 y 3

3. Ejemplo

líneas 4 y 5

4. Código

def + return

5. Test

assert



Funciones: De verdad

Del problema a la función

```
>>>sacarDigito(129345, 1)  
    ⇒5  
>>>sacarDigito(129345, 3)  
    ⇒3
```



Problema:

PROBLEMA DE LA CLASE 1

Dado un número de 6 dígitos, saque el 4.º dígito. **Indicación:** use `//`, `%` y variables auxiliares.

LA SOLUCIÓN, PASO A PASO

```
>>> numero = 129345
```

```
>>> superior = numero // 10000
```

```
⇒ 12
```

```
>>> resto = numero % 1000
```

```
⇒ 345
```

```
>>> numero = superior * 1000 + resto
```

```
⇒ 12345
```



¿Se acuerda del problema de la clase 1?

Dado un número de 6 dígitos, saque el 4.º dígito, usando `//`, `%` y variables auxiliares.

ANTES: A MANO, PARA UN CASO

```
>>> numero = 129345
>>> superior = numero // 10000
    ⇒ 12
>>> resto = numero % 1000
    ⇒ 345
>>> numero = superior * 1000 + resto
    ⇒ 12345
```

AHORA: ¿CÓMO SERÍA LA FUNCIÓN?

sacarDigito(numero, posicion)

Saca del número el dígito indicado por posición (≥ 1).

```
>>> sacarDigito(54321, 3)
    ⇒ 5421
>>> sacarDigito(54321, 4)
    ⇒ 5321
```



Problema 1 como función

Construya la función **sacarDigito(numero, posicion)**, que saca del número el dígito indicado por posición (≥ 1).

¿Cómo resolver el problema? Veamos caso por caso.

Si `posicion == 2`

se retorna

```
((numero // 100) * 10) + numero % 10
```

Si `posicion == 3`

se retorna

```
((numero // 1000) * 100) + numero % 100
```

Si `posicion == 4`

se retorna

```
((numero // 10000) * 1000) + numero % 1000
```



Problema 1 como función

¿CÓMO RESOLVER EL PROBLEMA?

La función **sacarDigito(numero, posicion)** saca el dígito de esa posición (≥ 1).

Si **posicion == 2**, se retorna:

$((\text{numero} // 100) * 10) + \text{numero} \% 10$

Si **posicion == 3**, se retorna:

$((\text{numero} // 1000) * 100) + \text{numero} \% 100$

Si **posicion == 4**, se retorna:

$((\text{numero} // 10000) * 1000) + \text{numero} \% 1000$

EL CASO A MANO

```
>>>numero = 129345
>>>superior = numero //
10000
=>12
>>>resto = numero % 1000
=>345
>>>numero = superior *
1000 + resto
=>12345
```



Problema 1 como función

Mismo patrón en los tres casos: cambia solo el **exponente**, que sale de **posicion**.

Si **posicion** == 2

```
((numero // 10**2) *  
10**1)  
+ numero % 10**1
```

Si **posicion** == 3

```
((numero // 10**3) *  
10**2)  
+ numero % 10**2
```

Si **posicion** == 4

```
((numero // 10**4) *  
10**3)  
+ numero % 10**3
```



Problema 1 como función

Mismo patrón en los tres casos: cambia solo el **exponente**, que sale de **posicion**.

Si **posicion** == 2

```
((numero // 10**2) *  
10**1)  
+ numero % 10**1
```

Si **posicion** == 3

```
((numero // 10**3) *  
10**2)  
+ numero % 10**2
```

Si **posicion** == 4

```
((numero // 10**4) *  
10**3)  
+ numero % 10**3
```



Problema 1 como función

Mismo patrón en los tres casos: cambia solo el **exponente**, que sale de **posicion**.

Si **posicion** == 2

```
((numero // 10**2) *  
10**1)  
+ numero % 10**1
```

Si **posicion** == 3

```
((numero // 10**3) *  
10**2)  
+ numero % 10**2
```

Si **posicion** == 4

```
((numero // 10**4) *  
10**3)  
+ numero % 10**3
```



Problema 1 como función

Caso genérico: si `posicion == k` (con $k \geq 1$), la fórmula es siempre la misma.

SE RETORNA

```
((numero // 10**k) * 10**(k-1))  
+ numero % 10**(k-1)
```

Ese `k` es justamente el parámetro `posicion`.



Problema 1 como función

```
# sacarDigito: int int -> int
# saca del numero el digito que
# indica posicion (>= 1)
# ejemplo: sacarDigito(54321, 3)
#         debe producir 5421
def sacarDigito(numero, posicion):
    assert posicion >= 1
    return ((numero // (10**posicion))
            * 10**(posicion-1))
            + numero % (10**(posicion-1))

# Test
assert sacarDigito(54321, 3) == 5421
```

LA RECETA, OTRA VEZ

Contrato, propósito, ejemplo, código y test: el mismo esquema de siempre.

El **assert** del cuerpo protege la restricción; el del final es la prueba.



Problema en parejas

Calcular la diferencia entre 2 instantes de tiempo. **Ejemplo:** entre 10:25 y 17:07 es 6:42.

Pista: una hora (10:25) se representa con 4 dígitos (**1025**).

minutos

De hora a minutos

1025 → 625

hhmm

De minutos a HHMM

625 → 1025

diferencia

Diferencia entre dos horas

1025 y 1707 → 642

¿Qué entregar? Las 3 funciones, cada una con su respectiva receta de diseño.

